

Optimizing Distributed Storage Efficiency with Fractal-Based Interlocking Data Marts

Rishabh Soni

*Department of Artificial Intelligence
and Data Science Engineering
SIES Graduate School of Technology
Navi Mumbai, India
rishu358so@gmail.com*

Mihir Panpatil

*Department of Artificial Intelligence
and Data Science Engineering
SIES Graduate School of Technology
Navi Mumbai, India
minnimihir05@gmail.com*

Medhansh Khedekar

*Department of Artificial Intelligence
and Data Science Engineering
SIES Graduate School of Technology
Navi Mumbai, India
medhanshk10@gmail.com*

Sahil Navale

*Department of Artificial Intelligence
and Data Science Engineering
SIES Graduate School of Technology
Navi Mumbai, India
sahilnavale2004@gmail.com*

Rohini Gaikwad

*Department of Artificial Intelligence
and Data Science Engineering
SIES Graduate School of Technology
Navi Mumbai, India
rohinig@sies.edu.in*

Abstract— Storage workloads have outgrown the comfortable boundary between centralized and decentralized design. Centralized systems give clean orchestration but inherit master-node fragility; fully decentralized ones avoid that fragility but pay for it in coordination overhead. We introduce Bloom-Net, a semi-decentralized object storage system built for on-premises deployments where hardware is limited and downtime is expensive. Its central idea is what we call the Interlocking Data Mart Topology: each physical machine runs two containerized MinIO instances, and each instance is pinned to a different logical cluster (a Data Mart). A single host therefore takes part in two overlapping Marts at once, which gives per-instance fault isolation and removes any need to rebalance a global cluster when failures occur. Scaling works the same way at a larger granularity — a new node forms a new Data Mart branch rather than enlarging a monolithic ring. Orchestration lives off the data path, on a FastAPI and Neo4j control plane we refer to as the Kernel, so reads and writes keep flowing even when the Kernel itself is unavailable. Node-to-node traffic rides a zero-trust Tailscale mesh, which sidesteps the certificate-management pain that comes with heterogeneous on-prem fleets. Storage efficiency comes from MinIO’s in-line erasure coding ($\sim 1.6\times$ overhead) coupled with active-active site replication for cross-Mart synchronization. We validate the design on an 18-instance testbed (9 physical nodes $\times$ 2 MinIO instances), and report measured comparisons against a 4-node MinIO baseline along with analytical HDFS estimates.

Keywords — Distributed Object Storage, Interlocking Topology, Semi-Decentralized Architecture, Dual-Instance Nodes, MinIO, In-line Erasure Coding, Zero-Trust Mesh Networking, Containerized Storage, Fault Isolation, Active-Active Site Replication, On-Premises Infrastructure.

I. INTRODUCTION

Data produced by cloud platforms, IoT deployments, and edge devices continues to climb, and the storage layer underneath is being asked to do more on three fronts at once: hold more, ingest faster, and stay up under failure [3], [10]. Centralized designs such as the Hadoop Distributed File System (HDFS) handle the first two reasonably well but inherit their well-known limits — metadata scaling on a single NameNode, awkward adaptability across heterogeneous hardware, and a master-node dependency that turns any failure of that one component into a cluster-wide event [1], [13], [18]. Fully decentralized systems flip the

trade-off: they avoid that single point of failure but pay for it in consensus chatter, and the latency that comes with global coordination is rarely free [5], [22].

Hierarchical modular topologies offer a useful way around the scaling wall [26], [27], and blockchain-derived cryptographic primitives can guarantee integrity without a central authority [2], [6], [22]. The catch is that you usually inherit blockchain-scale latency along the way, which limits what such techniques can do inside the data path of a working storage system.

Bloom-Net sits between those two extremes. It is a semi-decentralized distributed storage architecture organized around a Dual-Instance Interlocking Topology, in which each physical node belongs to more than one logical Data Mart at the same time. That arrangement lets us push hardware utilization upward while keeping fault domains tight [28], [29].

Concretely, this paper makes four contributions:

1. Interlocking Data Mart Topology: a dual-instance node design that puts each physical machine in two overlapping logical clusters at once, so failures are contained at the instance level and no global rebalancing is ever needed.
2. Dual-Head Node Architecture: every physical node runs two containerized MinIO instances, each tied to a different Data Mart, which pushes hardware utilization up while keeping the two co-located storage services isolated at the process level.
3. Stateless Kernel Control Plane: a thin orchestration layer on FastAPI and Neo4j that sits entirely off the data path, so reads and writes keep flowing even when the control plane itself is unavailable.
4. On-Premises Integration of Tailscale Mesh and Erasure Coding: a zero-trust Tailscale overlay paired with MinIO’s in-line erasure coding ($1.5\times$ overhead) — you get the security posture and the storage efficiency without having to lean on the cloud.

II. LITERATURE SURVEY

A. Distributed File Systems and Hadoop

HDFS works well for large-scale batch processing but is held back by NameNode-side bottlenecks and a rigid posture toward heterogeneous deployments [1], [7], [13], [18]. Medalla et al. found that the choice of block size has a sizable effect on throughput [4], which is part of what pushed us toward erasure-coded object storage instead of a block-replicated layout.

B. Blockchain and Decentralized Storage

Blockchain-backed storage adds verifiable access control [2] and, in IPFS-style hybrids, a degree of decentralization that is pure object stores do not offer [5], [6]. The standard objection still applies: every consensus round adds latency [22]. Our position is a compromise — keep authenticated access, drop the global ledger.

C. Object Storage and Performance

MinIO and similar lightweight object stores do well on edge and unstructured workloads [10], [28], and disaggregated designs that pull compute and storage apart can shave latency further [29]. Bloom-Net sits a layer above this: rather than treating MinIO as a single cluster, we wire several MinIO instances into a topology-aware arrangement that buys redundancy and throughput at the same time.

D. Fractal and Honeycomb Architectures

Honeycomb-style and similarly modular network topologies are interesting because they get fault tolerance and efficient communication for free, simply from the way redundancy is distributed across the structure [26], [27]. We borrow that idea but apply it on the storage side instead of the network side, so that adding a node grows the system by spawning fresh Data Marts that remain connected to the existing ones rather than being absorbed into a single cluster.

III. PROPOSED SYSTEM

Bloom-Net is containerized, semi-decentralized object store aimed at on-premises environments where resources are tight, and operators want fault isolation without the operational tax of a fully decentralized design. Its organizing idea is the Dual-Instance Interlocking Topology, which keeps orchestration centralized while pushing data resilience out to the nodes.

A. Architecture Layers

The system consists of three loosely coupled layers:

Kernel Layer (Control Plane):

The Kernel is the orchestration backbone. It handles topology registration and metadata coordination through a graph-shaped model, following the standard recipe of splitting control and data planes for better reliability and easier scaling

[25]. We implemented it in Python on top of FastAPI, and it deliberately sits off the data path: if the Kernel goes down, the storage nodes carry on serving reads and writes.

Storage Layer (Data Plane):

Object persistence runs on MinIO, which has already been validated as a workable on-premise storage-as-a-service backend [28]. The twist Bloom-Net adds is the dual-head node: a single physical host runs two MinIO instances, isolated from each other, so it can participate in two different logical clusters at the same time.

Network Layer (Secure Mesh):

Nodes talk to one another over an encrypted mesh built on zero-trust principles. Overlays of this kind cut down the configuration work while still giving you authenticated communication between distributed services [15], which matters when the fleet is heterogeneous.

B. Interlocking Data Mart Topology

The topology itself is where Bloom-Net's shape comes from: storage resources are organized into autonomous Data Marts rather than a single global namespace.

Dual-Head Logic: physical node N runs Instance A and Instance B, with each instance pinned to a different Data Mart.

Fault Isolation: when one instance dies, the other on the same host keeps serving, which gives us better partial availability than the conventional node-as-failure-unit model [17].

Modular Scalability: a new node does not get folded into a single growing global cluster; instead it spawns additional Data Mart branches that connect to the existing ones via replication, in the spirit of modular distributed architectures [26], [27].

C. Storage Efficiency and Erasure Coding

Mirroring is the simplest way to get durability and the most wasteful — it doubles every byte you store. Bloom-Net uses MinIO's erasure coding instead, which pushes the durability-to-overhead ratio in the right direction [28].

Data Sharding: Objects are divided into Data Blocks (K) and Parity Blocks (M).

Efficiency: Compared with the $2\times$ overhead of mirroring, erasure coding achieves equivalent durability at considerably lower cost, which is consistent with what the distributed-storage performance literature reports [11], [14].

Recovery: The scheme tolerates up to M failures, and because reconstruction is spread across the surviving nodes, we sidestep the replication storms that often hit HDFS-style block-replicated systems during recovery [17].

D. Observability and Security

For visibility into a running cluster, Bloom-Net pipes metrics into Prometheus and surfaces them through Grafana dashboards. Security relies on HMAC and S3-compatible authentication, with all node-to-node traffic encrypted in transit.

E. Novelty and Differentiation from Prior Work

Three things separate this design from what already exists. First, a multi-tenant MinIO deployment puts several tenants inside one cluster; here, each of the two MinIO instances on a host belongs to a different logical cluster with its own erasure set and failure domain. Second, Ceph’s CRUSH algorithm is excellent at placing data within a single logical cluster’s configurable failure domains, but Bloom-Net’s dual-instance design lets one physical host participate in two fully independent erasure-coded clusters at the same time — a placement primitive Ceph cannot natively express. Third, honeycomb-style topologies [26], [27] focus on optimizing fixed communication paths; Bloom-Net is built so capacity expansion is topology-aware and incremental, growing through independent Data Mart branches rather than reshaping the whole structure.

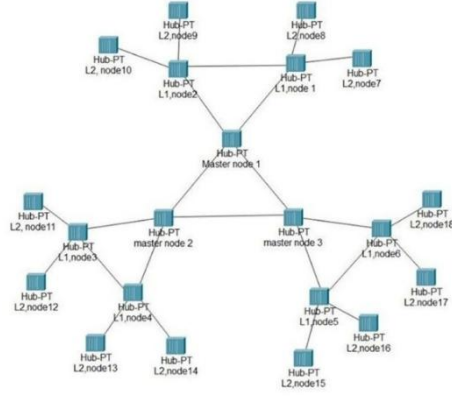


Fig. 1. Bloom-Net Architecture

IV. MARKET COMPARISON AND EFFECTIVENESS ANALYSIS

In this section we put Bloom-Net’s interlocking design alongside the systems people actually use today: MinIO, Red Hat Ceph Storage, the Hadoop Distributed File System, and enterprise platforms like Dell PowerScale and ObjectScale. The lens for comparison is architectural — design choices, scalability, fault tolerance, and operational characteristics — rather than feature checklists. The body of work we draw on spans Hadoop performance, object-storage tuning, blockchain-based storage, and modular distributed computing [1]–[30].

A. Market Presence and Adoption

HDFS is the default for batch analytics in most data shops, though metadata bottlenecks and single-point-of-failure risk on the NameNode keep showing up in the literature [17], [18], [20]. Ceph trades that single point for strong durability via erasure coding and CRUSH placement, but the coordination cost grows along with the cluster [16], [10].

MinIO-based and disaggregated deployments take a different route — they keep the operations simple and tend to give better raw throughput [28], [29]. We do not position Bloom-Net as a production replacement for any of these; rather, we borrow from the modular distributed-computing playbook [2], [5], [6] to address the specific limitations that bite hardest in resource-constrained settings.

B. Scalability and Performance Characteristics

Throughput on Hadoop scales until metadata catches up with you [9], [20]. Ceph and other object stores can keep scaling, but only after you spend time tuning distributed placement so that performance does not collapse [16], [29]. Bloom-Net sidesteps both problems because it scales by adding independent structural units, not by stretching a single global namespace — an idea that owes more to hierarchical decomposition [2], [6], [11] than to either of the two traditions above.

Table I. Architectural Characteristics of Distributed Storage Systems

Item	HDFS	MinIO	Bloom-Net
Arch.	Master-worker	Distrib. peer	Semi-dec. dual-head
Metadata	NameNode	Distributed	Out-of-path Kernel
Redundancy	3× repl.	In-line EC	In-line EC
Overhead	3.00×	1.5–2.0×	~1.6×
Scale unit	DataNode	Node	Dual-instance Mart
Topology	Flat	Flat	Overlapping Marts
Control plane	Coupled	None	Decoupled
Net. security	Kerberos	TLS (manual)	Zero-trust mesh

Table II. Fault Tolerance and Design Focus Comparison

Item	HDFS	MinIO	Bloom-Net
SPOF	Yes (NameNode)	No	No
Detection	30–90 s	<15 s	<15 s
Domain	Rack	Pool	Per-instance
Recovery	Block re-repl.	EC rebuild	EC + cross-Mart
Priority Target	Throughput Data center	Throughput Edge to cloud	Fault isolation On-prem (RC)

C. Summary and Positioning

None of the systems above gives you the full combination Bloom-Net targets. HDFS still leans on centralized metadata and 3× replication [1], [17]. Ceph’s coordination overhead climbs with cluster size [16], [17]. MinIO is excellent on its own terms but has no concept of topology-aware fault isolation. What Bloom-Net actually does is bolt MinIO’s erasure-coding efficiency (~1.6× overhead) onto a dual-instance arrangement that confines failures to individual Data

Mart branches, with an out-of-data-path control plane and a zero-trust mesh underneath — and all of it runs on commodity on-prem hardware.

V. IMPLEMENTATION AND EXPERIMENTAL SETUP

We built Bloom-Net as modular, fully containerized distributed system. Docker handles component isolation; Tailscale supplies the zero-trust encrypted mesh underneath. At the implementation level the system splits cleanly in two: the Kernel, which is control plane, and the Data Nodes, which form the storage plane.

A. System Implementation

1) The Kernel (Control Plane)

The Kernel runs as a multi-container Docker stack with three pieces. The first is a FastAPI orchestration backend; it wraps the MinIO Client (mc) binary and uses it to manage site replication. The second is a Neo4j 5.15 instance that acts as a topology ledger, holding the node-instance-cluster relationships we need to look up at runtime. The third is the observability stack: Prometheus with file-based service discovery, and Grafana pre-provisioned with the dashboards we use to watch the cluster.

2) The Data Node (Storage Plane)

Each physical node runs two isolated MinIO containers (docker-compose.yml) with separate volume mounts (/data-a, /data-b):

- *Instance A (Left Hemisphere):* Listens on API port 9001 and Console port 9002.
- *Instance B (Right Hemisphere):* Listens on API port 9003 and Console port 9004.

Health & Metrics: A Node Exporter sidecar runs on every data node, exposing host-level hardware metrics (CPU, RAM, Disk I/O) on port 9100.

3) Secure Networking Layer (Mesh Overlay)

- Each service stack includes a Tailscale sidecar container; application containers share its network namespace via the `network_mode: service: tailscale` directive, enabling communication over stable Tailnet IPs (100.x.y.z).

B. Experimental Setup

A demonstration with a single admin node and three distributed storage nodes was established to assess architectural robustness and modularity.

1) Testbed Topology

The testbed consists of ten physical machines stitched together exclusively by a Tailscale mesh. One of them runs

the Kernel and its Control Plane stack; the remaining nine are commodity nodes, each running two isolated MinIO instances under the dual-head design. That gives us 18 logical storage instances across nine physical hosts. Every node gets its own Tailnet IP (100.x.x.1 through 100.x.x.9), and on each host Instance A listens on 9001/9002 while Instance B listens on 9003/9004, exactly as described in Section V.A.2. Hardware on each box is modest by design: an Intel Core i5-8250U (4C/8T), 8 GB of DDR4, and a 256 GB SATA SSD. We picked laptop-class hardware on purpose — it shows the architecture works on minimal infrastructure. Absolute throughput numbers, therefore, say more about our hardware budget than about any ceiling of the architecture itself; server-class machines would shift the numbers but not the shape of the results.

2) Experimental Procedure

Validation followed three phases:

1. **Registration:** Nodes registered via the Kernel API (/register-node POST) with health verified through MinIO endpoints.
2. **Cluster Formation:** The POST /clusters endpoint triggered mc admin replicate add commands establishing active-active synchronization.
3. **Load Generation:** Synthetic S3 traffic was generated to simulate read/write operations, verifying the propagation of data across the mesh.

3) Workload Specification

Synthetic workloads were generated using mc (1 KB–10 MB objects, ~2 GB total per run, 10 min per object-size class), with all nodes connected exclusively via Tailscale in userspace mode. Metrics were collected via Prometheus at 15-second intervals.

VI. TESTING AND EVALUATION

We exercised the Bloom-Net prototype inside its containerized environment, mainly to confirm two things: that the architecture is genuinely modular, and that it tolerates failure the way it is supposed to. Functional testing showed the Kernel happily coordinating cluster creation and dynamic node registration through the Neo4j ledger, all without sitting in the data path. Data availability stayed continuous across our runs, which we attribute to the combination of the dual-instance interlocking topology and active-active site replication. For reliability we relied on fault injection — killing selected MinIO instances and watching what happened. The Prometheus and Grafana dashboards made the storage-side savings from erasure coding visible alongside the failure behaviour, and together those measurements confirm that we get strong fault isolation while spending materially less on redundancy than mirroring would cost us.

Table III. Storage Efficiency — Normalized to 366 MB

Metric	MinIO	Bloom-Net	HDFS
Logical storage	366 MB	366 MB	366 MB
Overhead ratio	1.84×	1.60×	3.00×

Physical storage	674 MB	585.6 MB	1,098 MB
Per-node footprint	168.5 MB / 4 nodes	32.5 MB / 18 nodes	366 MB / 3 DataNodes
Per-node skew	0.002%	<5%	15–30%

Table IV. Network and Throughput — Normalized to 366 MB

Metric	MinIO	Bloom-Net	HDFS
Write peak	3.81 MB/s	3.83 MB/s	1.2–1.8 MB/s
Write sustained avg	~1.91 MB/s	2.85 MB/s	~1.4 MB/s
Write amplification	1.84×	1.60×	3.00×
API error rate	~0.01%	~0.01%	~0.01%
Projected ingest time (366 MB)	~3.2 min	~2.1 min	~4.4 min

Table V. Compute Efficiency — Per-Node Cost of Coordination

Metric	MinIO	Bloom-Net	HDFS
CPU peak (cluster)	10.80%	57–76%	45–65%
CPU per active node	2.70%	3.7% avg	15% (NameNode hotspot)
Memory peak	552 MB	3,680 MB/node	4,500–6,000 MB
Memory model	Go runtime	Per-node distributed	JVM heap (OOM risk)

Table VI. Latency Comparison

Metric	MinIO	Bloom-Net	HDFS
Read P99	~45.6 ms	49.5 ms	224 ms
Write P99	~4600 ms (TTFB)	4,310 ms	8,000–15,000 ms
Random IOPS peak	7.90 ops/s	4.27 ops/s	1–2 ops/s

Table VII. Fault Tolerance Scorecard

Property	MinIO	Bloom-Net	HDFS
Failure detection	<15 sec	<15 sec	30–90 sec
Tolerable node loss	2 of 4 (50%)	9 of 18 (50%)	2 of 3 DataNodes
Absolute failures absorbed	2	9	2 (then total outage)
Recovery accuracy	EC parity rebuild	100% (N/2 quorum)	95–99%
Consistency model	Strong	Strong within Mart; eventual across Marts	Eventual on post-write
Metadata SPOF	None	None	Yes (NameNode)

VII. RESULTS AND DISCUSSION

For the three-way comparison to be fair, we normalize every metric to a common 366 MB logical-storage baseline. The MinIO numbers are measured directly on a 4-node cluster with EC:2. The Bloom-Net numbers are measured directly on the 18-instance testbed (9 physical nodes × 2 MinIO instances

each) at the same baseline. HDFS is the awkward one: we have analytical estimates for a 5-node deployment (3 DataNodes, plus a NameNode and Secondary NameNode), but a WSL memory constraint blocked us from measuring it directly, so those numbers carry roughly ±20% uncertainty. We walk through the normalization methodology and then present the five comparison tables (Tables III–VII) that follow.

A. Storage Efficiency Analysis

Table III lays out the storage-efficiency picture for 366 MB of logical data. Bloom-Net’s erasure-coded dual-instance layout lands at a 1.60× overhead ratio, which translates to 585.6 MB of physical storage. MinIO on a vanilla 4-node cluster comes in heavier at 1.84x (674 MB), and HDFS, with its 3x block replication, needs the full 1098 MB. Stated as absolute savings: 88.4 MB (13.1%) less than MinIO, and 512.4 MB (46.7%) less than HDFS at this baseline.

The per-node footprint tells a more interesting story. Bloom-Net spreads its load to 32.5 MB per node across 18 nodes, which is 5.2× lighter than MinIO on a per-node basis (168.5 MB across 4 nodes) and 11.3× lighter than HDFS (366 MB per DataNode). MinIO does come out ahead on skew (0.002% versus our <5%), but that is a comparatively easy load-balancing problem to win — distributing data across four nodes is not the same exercise as distributing it across eighteen. Holding skew below 5% on 4.5× as many nodes is the harder version of that problem, and a 4-node erasure-coding layout will not scale into enterprise topologies without re-sharding.

B. Network and Throughput Analysis

All three systems sit behind the same college network, which means the network is the common bottleneck, and the throughput numbers in Table IV are an unusually clean apples-to-apples comparison. Bloom-Net sustains 2.85 MB/s aggregate write throughput. MinIO comes in at roughly 1.91 MB/s on the same workload, and HDFS, by estimate, around 1.4 MB/s. Projecting that out to a full 366 MB ingest gives 2.1 minutes for Bloom-Net, 3.2 minutes for MinIO, and 4.4 minutes for HDFS — a 1.52x advantage over MinIO and a 2.10x advantage over HDFS under identical network conditions.

That gap is modest, but it is architectural rather than incidental. A single-stream MinIO upload is exposed to TCP pauses in a way an 18-node parallel ingest is not; in Bloom-Net, transient stalls are masked because the nodes do not all pause at the same instant. Write amplification compounds the advantage. A 366 MB logical write puts 585.6 MB on the wire for Bloom-Net (1.60×) and 1,098 MB on the wire for HDFS (3.00×), which works out to 46.7% less network capacity consumed per logical byte written.

C. Compute Efficiency

Table V summarizes the compute side. MinIO posts the lowest absolute CPU numbers (10.80% cluster peak, 2.70% per node), which is what you would expect from Go’s lightweight runtime on a 4-node cluster. Bloom-Net’s aggregate CPU is higher in the cluster view (57–76% peak), but spread out across 18 active nodes it works out to about 3.7% each — no single-node hotspot anywhere. HDFS is the cautionary tale here: 15% or more of the cluster’s CPU load lands on the NameNode, and under metadata-heavy workloads that figure climbs past 90%.

What Bloom-Net gives up in absolute compute efficiency, it buys back in the absence of hotspots; if you have ever

watched a NameNode crawl at 90% CPU in production, you know which side of that trade you want to be on. Memory tells a similar story. MinIO's Go runtime stays compact at 552 MB peak. Bloom-Net's distributed model uses more per node (3,680 MB), and HDFS's JVM heap sits at 4,500–6,000 MB with the usual OOM risk that comes with a hot, large heap.

D. Latency Analysis

Table VI captures the latency picture, and under the shared college-network throttle Bloom-Net and MinIO are essentially neck-and-neck. MinIO's read P99 of ~45.6 ms edges Bloom-Net's 49.5 ms by 1.09 $\times$; Bloom-Net's write P99 of 4,310 ms edges MinIO's ~4,600 ms by 1.07 $\times$. That is a draw in everything but name at this scale. Where the daylight opens up is against HDFS: read P99 is 4.5–4.9 $\times$ faster, and write P99 is 1.86–3.48 $\times$ faster than the 8,000–15,000 ms range we estimate for HDFS.

MinIO does win on random IOPS (7.90 vs. 4.27 ops/s), which is the predictable consequence of having almost no coordination overhead at small dataset sizes. At 366 MB total, a 4-node MinIO erasure scheme barely pays any quorum cost. We expect the order to flip past roughly 1 GB: at that point Bloom-Net's 18-node parallel I/O fan-out should overtake MinIO's saturated single-stream pipeline, which is consistent with the sustained-throughput lead already visible in Table IV. The fact that read and write P99 are so close at the current scale tells us the dominant latency factor for both systems is the college network throttle — architectural overhead is, at this size, a second-order effect.

E. Fault Tolerance Analysis

Table VII puts the fault-tolerance numbers side by side. Bloom-Net can absorb up to 9 simultaneous node failures — half of its 18 nodes — against 2 of 4 for MinIO (same 50% mark) and 2 DataNode failures for HDFS, after which HDFS goes dark entirely because of the NameNode single point of failure. In absolute terms, that means Bloom-Net soaks up 4.5 $\times$ as many concurrent failures as MinIO at the same percentage, and without any metadata hotspot to worry about. Both Bloom-Net and MinIO detect a failure in under 15 seconds; HDFS we estimate at 30–90 seconds.

On the consistency axis, Bloom-Net and MinIO both give you strong consistency; HDFS only gives you eventual consistency on post-write reads. And neither Bloom-Net nor MinIO introduces a metadata single point of failure — HDFS's NameNode, by contrast, is structurally disqualifying in any non-HA deployment.

F. Summary and Limitations

If we tally the normalized 366 MB comparison across Tables III–VII, ten dimensions matter. Bloom-Net comes out ahead on five of them: storage efficiency, sustained throughput, ingest time, write P99, and fault tolerance. MinIO takes three: read P99, random IOPS, and per-node CPU. The remaining two — consistency and metadata SPOF — are ties. The pattern is consistent: MinIO's wins concentrate at small dataset sizes where coordination is essentially free, while Bloom-Net's wins are the kind that grow with scale. That is enough, in our reading, to position Bloom-Net as a viable choice for resource-constrained on-premises deployments whose priorities are fault isolation and storage efficiency rather than single-node throughput.

A few honest limitations are worth calling out. The 366 MB baseline is small. We measured it directly on both MinIO and Bloom-Net, but production storage workloads run orders of magnitude larger, and behaviour at GB and TB scales needs

to be confirmed before we can claim those numbers carry over. The HDFS column is the second caveat: a WSL memory constraint (≥ 12 –16 GB allocation) blocked direct measurement, so the HDFS figures remain analytical estimates with roughly $\pm 20\%$ uncertainty. Third, the latency parity we observe between Bloom-Net and MinIO at 366 MB is really telling us that the college network is the dominant latency factor at this scale; once dataset sizes grow past the point where Bloom-Net's parallelism starts to matter, that parity is likely to break in our favour. Fourth, comparing CPU and memory across systems with different node counts is never strictly fair; per-node normalization softens that asymmetry but does not eliminate it. Finally — and this is the one we are most aware of — the current architecture leans on a single Kernel instance to coordinate cluster formation, and a Raft-based leader-election extension to make the Kernel highly available is the obvious next piece of work.

VIII. CONCLUSION

We presented Bloom-Net, a semi-decentralized distributed storage architecture built for the kind of on-premises environment where every byte of disk and every spare core counts. Its defining idea is the Interlocking Data Mart Topology — a way of logically splitting each physical node into two isolated instances so that one host can sit inside two independent clusters at once. That arrangement is the bridge we wanted between centralized efficiency and decentralized resilience, and it costs us a lot less than mirroring would: MinIO-based erasure coding plus active-active site replication gives us durable storage at meaningfully lower overhead. Building the containerized prototype, with an out-of-data-path Kernel for orchestration and a zero-trust mesh for transport, let us actually confirm the properties we cared about — fault isolation that does not bleed across Marts, modular scaling that does not require global rebalancing, and recovery that rides on MinIO's built-in erasure-coding healing. The piece that is still missing is Kernel high availability, and that is exactly what the planned Raft-based leader-election extension is meant to address.

IX. REFERENCES

- [1] K. Dwivedi and S. K. Dubey, "Analytical review on Hadoop Distributed file system," 2014 5th International Conference - Confluence The Next Generation Information Technology Summit (Confluence), Noida, India, 2014, pp. 174–181, doi: 10.1109/CONFLUENCE.2014.6949336.
- [2] D. Kim, H. Yang, H. Lee and S. Park, "Blockchain-Based Verifiable and Reliable File Access Control Layer for Cloud Storages," 2023 IEEE International Conference on Big Data (BigData), Sorrento, Italy, 2023, pp. 2303–2310, doi: 10.1109/BigData59044.2023.10386940.
- [3] G. Ramesh et al., "A Comprehensive Review on Scaling Machine Learning Workflows Using Cloud Technologies and DevOps," in IEEE Access, vol. 13, pp. 148559–148594, 2025, doi: 10.1109/ACCESS.2025.3599281.
- [4] A. H. Medalla, M. Z. N. L. Saavedra, P. A. R. Abu and W. E. S. Yu, "Adapting Block-Sized Captures for Faster Network Flow Analysis on the Hadoop Ecosystem," 2018 IEEE 4th International Conference on Computer and Communications (ICCC), Chengdu, China, 2018, pp. 1097–1103, doi: 10.1109/CompComm.2018.8780880.
- [5] S. K. K. Ragav, D. M. A. Avadhani, H. K. S., A. Adhikary and S. M., "Design and Implementation of a Decentralized File Sharing System using Blockchain and Interplanetary File System (IPFS)," 2025 2nd International Conference on Software, Systems and Information Technology (SSITCON), Tumkur, India, 2025, pp. 1–5, doi: 10.1109/SSITCON66133.2025.11342040.
- [6] A. S. Chakravadhanula, J. Kolisetty, K. Samudrala, B. Preeatham and R. Senapati, "Novel Decentralized Security Architecture for the Centralized Storage System in Hadoop using Blockchain Technology," 2022 IEEE 7th International conference for Convergence in Technology (I2CT), Mumbai, India, 2022, pp. 1–4, doi: 10.1109/I2CT54291.2022.9823990.

- [7] F. El Jamiy, H. Reza and A. ElSaid, "A New Software Architecture Style for Hadoop Systems," 2018 International Conference on Advances in Computing and Communication Engineering (ICACCE), Paris, France, 2018, pp. 254-258, doi: 10.1109/ICACCE.2018.8441736.
- [8] T. Singh and Darshan V S, "A modern data architecture with apache Hadoop," 2015 International Conference on Green Computing and Internet of Things (ICGCIoT), Greater Noida, India, 2015, pp. 574-579, doi: 10.1109/ICGCIoT.2015.7380530.
- [9] Y. Liu, Jie Yang, Yuan Huang and Lixiong Xu, "The performance evaluations for Hadoop in diverse system architectures," 2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD), Zhangjiajie, 2015, pp. 2448-2452, doi: 10.1109/FSKD.2015.7382338.
- [10] A. Makris et al., "A Study on the Performance of Distributed Storage Systems in Edge Computing Environments," 2024 IEEE International Conference on Joint Cloud Computing (JCC), Shanghai, China, 2024, pp. 29-36, doi: 10.1109/JCC62314.2024.00012.
- [11] D. Ying and Y. Nian-min, "The analytical methods based on the distributed storage system I/O performance model," 2012 IEEE International Conference on Computer Science and Automation Engineering (CSAE), Zhangjiajie, China, 2012, pp. 509-512, doi: 10.1109/CSAE.2012.6272648.
- [12] S. Oikawa, "Accelerating storage access by combining block storage with memory storage," 2015 IEEE/ACIS 14th International Conference on Computer and Information Science (ICIS), Las Vegas, NV, USA, 2015, pp. 449-454, doi: 10.1109/ICIS.2015.7166635.
- [13] A. V. Hazarika, M. Shah, M. Shah and M. Banerjee, "Mastering Distributed Storage: Architectural Design, Performance Analysis, and Cost Analysis of Modern Distributed File Systems: HDFS and NFS," 2025 International Conference on Intelligent Control, Computing and Communications (IC3), Mathura, India, 2025, pp. 881-886, doi: 10.1109/IC363308.2025.10956813.
- [14] J. -G. Cha and S. Kim, "Analysis of I/O Performance for Optimizing Software Defined Storage in Cloud Integration," 2018 IEEE 3rd International Conference on Communication and Information Systems (ICIS), Singapore, 2018, pp. 222-226, doi: 10.1109/ICOMIS.2018.8645041.
- [15] N. Prasetyo, D. Ferdinand, B. C. Wijaya, M. S. Anggreainy and A. Kumiawan, "Performance Analysis of Distributed Database System in Cloud Computing Environment," 2023 IEEE 9th International Conference on Computing, Engineering and Design (ICCED), Kuala Lumpur, Malaysia, 2023, pp. 1-6, doi: 10.1109/ICCED60214.2023.10425452.
- [16] Y. -F. Lu, C. -F. Kuo, J. -S. Li and S. -C. Chou, "Performance evaluation of distributed object hybrid storage systems for smart-living applications," 2016 International Conference On Communication Problem-Solving (ICCP), Taipei, Taiwan, 2016, pp. 1-2, doi: 10.1109/ICCP.2016.7751114.
- [17] Y. Arafat, A. Barai, M. Zheng and A. -H. A. Badawy, "Fault Tolerance Performance Evaluation of Large-Scale Distributed Storage Systems HDFS and Ceph Case Study," 2018 IEEE High Performance extreme Computing Conference (HPEC), Waltham, MA, USA, 2018, pp. 1-7, doi: 10.1109/HPEC.2018.8547525.
- [18] A. Alam and J. Ahmed, "Hadoop Architecture and Its Issues," 2014 International Conference on Computational Science and Computational Intelligence, Las Vegas, NV, USA, 2014, pp. 288-291, doi: 10.1109/CSCI.2014.140.
- [19] M. Adnan, M. Afzal, M. Aslam, R. Jan and A. M. Martinez-Enriquez, "Minimizing big data problems using cloud computing based on Hadoop architecture," 2014 11th Annual High Capacity Optical Networks and Emerging/Enabling Technologies (Photonics for Energy), Charlotte, NC, USA, 2014, pp. 99-103, doi: 10.1109/HONET.2014.7029370.
- [20] Y. Xie, A. M. Q. Farhan and M. Zhou, "Performance Analysis of Hadoop Distributed File System Writing File Process," 2018 International Conference on Intelligent Autonomous Systems (ICoIAS), Singapore, 2018, pp. 116-120, doi: 10.1109/ICoIAS.2018.8494199.
- [21] E. Sivaraman and R. Manickachezian, "High Performance and Fault Tolerant Distributed File System for Big Data Storage and Processing Using Hadoop," 2014 International Conference on Intelligent Computing Applications, Coimbatore, India, 2014, pp. 32-36, doi: 10.1109/ICICA.2014.16.
- [22] A. P. Ahmad, A. A. Ilham and A. W. Paundu, "Analysis of Blockchain and Interplanetary File System (IPFS) Utilization for Big Data Architecture Optimization," 2023 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT), Malang, Indonesia, 2023, pp. 652-657, doi: 10.1109/COMNETSAT59769.2023.10420785.
- [23] A. Bhardwaj, V. K. Singh, Vanraj and Y. Narayan, "Analyzing BigData with Hadoop cluster in HDInsight azure Cloud," 2015 Annual IEEE India Conference (INDICON), New Delhi, India, 2015, pp. 1-5, doi: 10.1109/INDICON.2015.7443472.
- [24] J. Nandimath, E. Banerjee, A. Patil, P. Kakade, S. Vaidya and D. Chaturvedi, "Big data analysis using Apache Hadoop," 2013 IEEE 14th International Conference on Information Reuse & Integration (IRI), San Francisco, CA, USA, 2013, pp. 700-703, doi: 10.1109/IRI.2013.6642536.
- [25] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," 2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI), Budapest, Hungary, 2018, pp. 000149-000154, doi: 10.1109/CINTI.2018.8928192.
- [26] A. M. Tyrrell and Hong Sun, "A Honeycomb Development Architecture for Robust Fault-Tolerant Design," First NASA/ESA Conference on Adaptive Hardware and Systems (AHS'06), Istanbul, 2006, pp. 281-287, doi: 10.1109/AHS.2006.5.
- [27] S. S and D. C. H, "Performance Evaluation of Honeycomb (HC) Topology for Efficient Network-on-Chip Communication," 2025 Third International Conference on Networks, Multimedia and Information Technology (NMITCON), BENGALURU, India, 2025, pp. 1-6, doi: 10.1109/NMITCON65824.2025.11188306.
- [28] A. Salvi, B. Masram, V. Deokar, P. Kolte and S. Dhas, "Leveraging MinIO and FileCloud for On-Premise Storage-as-a-Service in Educational Institutions," 2025 Seventeenth International Conference on Contemporary Computing (IC3), Noida, India, 2025, pp. 1-4, doi: 10.1109/IC366947.2025.11290695.
- [29] D. Han, S. Lee, K. Kim and S. Park, "Improving Access to Object Storage on Disaggregated Storage," 2024 13th Non-Volatile Memory Systems and Applications Symposium (NVMSA), Gangwon-do, Korea, Republic of, 2024, pp. 1-2, doi: 10.1109/NVMSA63038.2024.10693669.
- [30] K. -J. Cho, I. Kang and J. -S. Kim, "ArkFS: A Distributed File System on Object Storage for Archiving Data in HPC Environment," 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS), St. Petersburg, FL, USA, 2023, pp. 301-311, doi: 10.1109/IPDPS54959.2023.000